

Sentiment140__Tweet__Sentiment__Analysis

September 15, 2026

1 Sentiment Analysis of Tweets using the Sentiment140 Dataset

This notebook builds a machine learning project for classifying tweet sentiment as negative 0, neutral 2, or positive 4 using Sentiment140-style data.

Dataset note: the most common Sentiment140 dataset contains only negative 0 and positive 4 labels. If your CSV does not contain neutral 2 labels, the notebook will train a binary model. If you use a version with neutral labels, the same workflow supports all three classes.

1.1 1. Import Libraries

```
[ ]: import html
import json
import re
import string
from collections import Counter
from pathlib import Path

import joblib
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, \
    confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline

try:
    from wordcloud import WordCloud
except ImportError:
    WordCloud = None
```

```
sns.set_theme(style="whitegrid")
```

1.2 2. Load the Dataset

Place the full Sentiment140 CSV in the data folder. The common filename is training.1600000.processed.noemoticon.csv.

This notebook also includes data/sample_tweets.csv so the workflow can run immediately.

```
[ ]: DATA_PATH = Path("data/sample_tweets.csv")
# For the full dataset, use:
# DATA_PATH = Path("data/training.1600000.processed.noemoticon.csv")

EXPECTED_COLUMNS = ["target", "id", "date", "query", "user", "text"]

def load_sentiment140(path, sample_size=None):
    df = pd.read_csv(path, encoding="latin-1")
    if "target" not in df.columns or "text" not in df.columns:
        df = pd.read_csv(path, encoding="latin-1", header=None,
names=EXPECTED_COLUMNS)
    df = df[["target", "text"]].dropna()
    if sample_size and sample_size < len(df):
        df = df.sample(sample_size, random_state=42)
    return df

df = load_sentiment140(DATA_PATH)
df.head()
```

```
[ ]: df.info()
df["target"].value_counts().sort_index()
```

1.3 3. Data Preprocessing

Tweets usually contain URLs, mentions, hashtags, punctuation, emojis, and inconsistent spacing. The function below cleans text into a simpler representation for machine learning.

```
[ ]: URL_RE = re.compile(r"https?://\S+|www\.\S+")
MENTION_RE = re.compile(r"@w+")
HASHTAG_RE = re.compile(r"#(\w+)")
NON_ALPHA_RE = re.compile(r"[^a-zA-Z\s]")
MULTISPACE_RE = re.compile(r"\s+")

NEGATION_MAP = {
    "can't": "can not",
    "cannot": "can not",
    "won't": "will not",
    "n't": " not",
    "i'm": "i am",
```

```

    "it's": "it is",
    "that's": "that is",
    "what's": "what is",
    "there's": "there is",
    "you're": "you are",
    "they're": "they are",
    "we're": "we are",
    "i've": "i have",
    "don't": "do not",
    "didn't": "did not",
}

def clean_tweet(text):
    value = html.unescape(str(text)).lower()
    value = URL_RE.sub(" ", value)
    value = MENTION_RE.sub(" ", value)
    value = HASHTAG_RE.sub(r"\1", value)
    for src, dst in NEGATION_MAP.items():
        value = value.replace(src, dst)
    value = value.translate(str.maketrans("", "", string.punctuation))
    value = NON_ALPHA_RE.sub(" ", value)
    value = MULTISPACE_RE.sub(" ", value).strip()
    return value

def label_name(label):
    return {0: "negative", 2: "neutral", 4: "positive", "0": "negative", "2": "neutral", "4": "positive"}.get(label, str(label))

df["clean_text"] = df["text"].apply(clean_tweet)
df["sentiment"] = df["target"].apply(label_name)
df = df[df["clean_text"].str.len() > 0].copy()
df.head()

```

1.4 4. Exploratory Data Analysis

```

[ ]: plt.figure(figsize=(7, 4))
order = [label for label in ["negative", "neutral", "positive"] if label in
        ↪set(df["sentiment"])]
sns.countplot(data=df, x="sentiment", order=order)
plt.title("Sentiment Distribution")
plt.xlabel("Sentiment")
plt.ylabel("Tweet Count")
plt.show()

```

```

[ ]: df["tweet_length"] = df["clean_text"].str.split().apply(len)

plt.figure(figsize=(8, 4))

```

```
sns.histplot(data=df, x="tweet_length", hue="sentiment", bins=20, kde=True)
plt.title("Tweet Length Distribution by Sentiment")
plt.xlabel("Number of Words")
plt.show()
```

```
[ ]: STOP_WORDS = {
    "the", "a", "an", "and", "or", "is", "are", "was", "were", "to", "of", "in",
    "for", "on", "with", "this", "that", "it", "my", "i", "you", "me", "at",
}

def top_words_by_sentiment(data, sentiment, n=20):
    words = " ".join(data.loc[data["sentiment"] == sentiment, "clean_text"]).
    ↪split()
    counts = Counter(word for word in words if word not in STOP_WORDS)
    return pd.DataFrame(counts.most_common(n), columns=["word", "count"])

for sentiment in order:
    top_words = top_words_by_sentiment(df, sentiment)
    if top_words.empty:
        continue
    plt.figure(figsize=(8, 5))
    sns.barplot(data=top_words, x="count", y="word")
    plt.title(f"Top Words in {sentiment.title()} Tweets")
    plt.xlabel("Frequency")
    plt.ylabel("")
    plt.show()
```

```
[ ]: if WordCloud is None:
    print("Install wordcloud to generate word clouds: pip install wordcloud")
else:
    for sentiment in order:
        text = " ".join(df.loc[df["sentiment"] == sentiment, "clean_text"])
        if not text.strip():
            continue
        cloud = WordCloud(width=1000, height=600, background_color="white",
        ↪collocations=False).generate(text)
        plt.figure(figsize=(10, 6))
        plt.imshow(cloud, interpolation="bilinear")
        plt.axis("off")
        plt.title(f"Word Cloud: {sentiment.title()}")
        plt.show()
```

1.5 5. Feature Engineering and Train-Test Split

The models below use CountVectorizer and TF-IDF. Both convert text into numerical features that machine learning algorithms can learn from.

```
[ ]: X = df["clean_text"]
y = df["sentiment"]

stratify = y if y.value_counts().min() > 1 else None
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=stratify,
)

X_train.shape, X_test.shape
```

1.6 6. Model Development

This section trains three baseline models:

- Logistic Regression with TF-IDF
- Naive Bayes with CountVectorizer
- Random Forest with TF-IDF

```
[ ]: models = {
    "Logistic Regression + TF-IDF": Pipeline([
        ("features", TfidfVectorizer(max_features=100000, ngram_range=(1, 2),
        ↪min_df=1)),
        ("model", LogisticRegression(max_iter=1000, class_weight="balanced")),
    ]),
    "Naive Bayes + CountVectorizer": Pipeline([
        ("features", CountVectorizer(max_features=80000, ngram_range=(1, 2),
        ↪min_df=1)),
        ("model", MultinomialNB()),
    ]),
    "Random Forest + TF-IDF": Pipeline([
        ("features", TfidfVectorizer(max_features=30000, ngram_range=(1, 2),
        ↪min_df=1)),
        ("model", RandomForestClassifier(n_estimators=120, random_state=42,
        ↪class_weight="balanced")),
    ]),
}

results = []
trained_models = {}

for name, model in models.items():
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)
    accuracy = accuracy_score(y_test, predictions)
```

```

        results.append({"model": name, "accuracy": accuracy})
        trained_models[name] = model

results_df = pd.DataFrame(results).sort_values("accuracy", ascending=False)
results_df

```

1.7 7. Model Evaluation

```

[ ]: best_model_name = results_df.iloc[0]["model"]
      best_model = trained_models[best_model_name]
      y_pred = best_model.predict(X_test)

      print("Best model:", best_model_name)
      print("Accuracy:", accuracy_score(y_test, y_pred))
      print("\nClassification Report:\n")
      print(classification_report(y_test, y_pred, zero_division=0))

```

```

[ ]: labels = sorted(y.unique())
      cm = confusion_matrix(y_test, y_pred, labels=labels)

      plt.figure(figsize=(6, 5))
      sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=labels,
                  yticklabels=labels)
      plt.title("Confusion Matrix")
      plt.xlabel("Predicted")
      plt.ylabel("Actual")
      plt.show()

```

1.8 8. Save the Best Model

```

[ ]: Path("models/sentiment_model").mkdir(parents=True, exist_ok=True)
      Path("reports").mkdir(exist_ok=True)

      joblib.dump(best_model, "models/sentiment_model/sentiment_model.joblib")

      metrics = {
          "best_model": best_model_name,
          "accuracy": float(accuracy_score(y_test, y_pred)),
          "all_model_results": results_df.to_dict(orient="records"),
      }

      with open("reports/notebook_metrics.json", "w", encoding="utf-8") as f:
          json.dump(metrics, f, indent=2)

      metrics

```

1.9 9. Predict Sentiment for New Tweets

```
[ ]: def predict_sentiment(tweet):
    cleaned = clean_tweet(tweet)
    prediction = best_model.predict([cleaned])[0]
    return prediction

examples = [
    "I love this new phone update, it works perfectly!",
    "This service is awful and I am very disappointed.",
    "The meeting starts at 4 PM today.",
]

for tweet in examples:
    print(tweet, "->", predict_sentiment(tweet))
```

1.10 10. Conclusion

This notebook demonstrates a complete machine learning workflow for tweet sentiment analysis:

- Cleaned noisy tweet text
- Explored sentiment distribution and common words
- Converted text into numerical features using CountVectorizer and TF-IDF
- Trained Logistic Regression, Naive Bayes, and Random Forest models
- Evaluated models using accuracy, classification report, and confusion matrix
- Saved the best model for reuse

For larger experiments, run the notebook on a larger sample or the full Sentiment140 dataset. For advanced performance, compare these baselines with LSTM, BERT, or RoBERTa-based models.